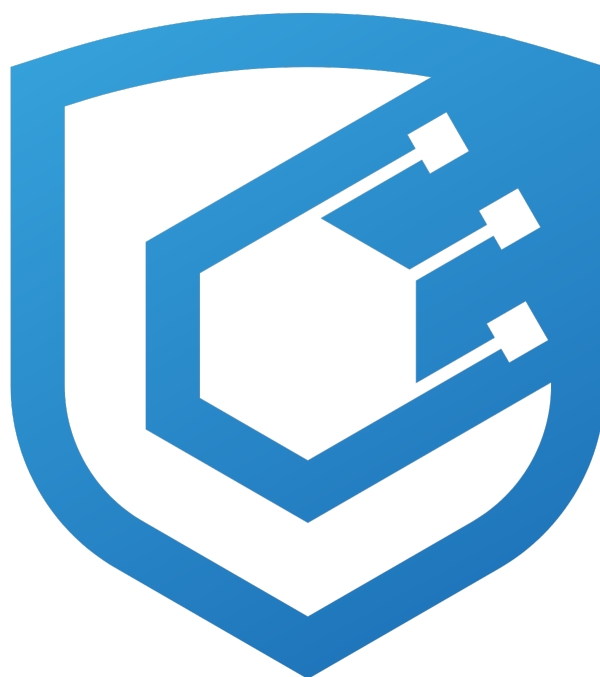




Stake.link PR 187 Audit Report

Prepared by [Cyfrin](#)

Version 2.0

Lead Auditors

[SanadSquad](#)

September 29, 2026

Contents

1	About Cyfrin	2
2	Disclaimer	2
3	Risk Classification	2
4	Protocol Summary	2
5	Audit Scope	2
6	Executive Summary	3
6.1	Centralization Risks	3
6.2	Defensive Improvements	3
6.3	Protocol Invariants	3
7	Findings	5
7.1	Informational	5
7.1.1	Dead code in StakingPool addFee and updateFee after the removal of the strategy rewards update	5
7.1.2	Lack of _feeBasisPoints non zero validation in addFee	6
7.1.3	Unchecked uint256 cast in _updateStrategyRewards silently corrupts totalStaked if neg- ative rewards ever exceed it.	7
7.1.4	Lack of _receiver zero address validation in addFee and updateFee halts all reward updates until the owner corrects the entry	8

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited. Unless the scope explicitly notes otherwise, the review is limited to the application layer and does not extend to the correctness of API/SDK/execution environment implementations, compilers, standard and third-party libraries, underlying operating systems and hardware.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

stake.link is a liquid staking protocol that lets users stake assets across Chainlink, Polygon and Espresso node operators, receiving a rebasing liquid staking token backed 1:1 by the underlying stake. Staked assets are held by StakingPool, which routes deposits into a set of pluggable strategy contracts and periodically rebases by polling each strategy for its deposit change, then minting and distributing fees on any newly earned rewards.

Each strategy exposes an owner-controlled list of fee receivers and basis point amounts, charged against rewards at rebase time. Previously, adding or updating a fee first triggered a full rebase across every strategy, so that outstanding rewards were credited at the old rate before the new rate took effect. PR 187 removes that forced rebase from the fee setters in StakingPool, EspressoStrategy, PolygonStrategy and VaultControllerStrategy, leaving fee changes to take effect at the next scheduled rebase.

This change improves the resilience of the overall system. Because adding or updating a fee receiver no longer forces a call into StakingPool::_updateStrategyRewards, a misconfigured fee entry no longer blocks the transaction that would correct it. Under the previous design, an invalid configuration such as a zero address receiver would cause every reward update to revert, and the corrective call would revert along with them whenever rewards were positive, leaving the protocol recoverable only through an upgrade. The configuration can now be revoked directly.

The trade-off is that reward attribution is no longer exact. A fee receiver added between rebases accrues a share of rewards that were earned before it was configured, and a rate change applies retroactively to rewards that accrued under the previous rate. This is largely mitigated operationally by having the rebaseController call StakingPool::updateStrategyRewards immediately before any fee configuration change.

5 Audit Scope

The audit scope was changes to [PR187](#) limited to the following files:

```
contracts/core/StakingPool.sol
contracts/espressoStaking/EsspressoStrategy.sol
contracts/linkStaking/base/VaultControllerStrategy.sol
contracts/polygonStaking/PolygonStrategy.sol
```

6 Executive Summary

Over the course of 2 days, the Cyfrin team conducted an audit on the [PR 187](#) code provided by [Stake.link](#). The findings consist of 4 Informational.

6.1 Centralization Risks

As with any protocol that custodies user funds in smart contracts, stake.link users place a degree of trust in the protocol administrators. It is worth stating plainly what that trust covers. The staking pool and every strategy are UUPS upgradeable with `_authorizeUpgrade` gated on the owner, so the owner can replace the implementation of any contract holding user funds. `StakingPool::addStrategy` also grants an unlimited token approval to the strategy address it is given. Either path would allow a compromised or malicious owner key to reach staked assets, and this is inherent to the upgradeable design rather than anything introduced by this PR.

Outside of those paths, the owner's routine powers do not include moving user funds. Deposits and withdrawals are gated on the priority pool rather than the owner, and the owner-facing surface is limited to configuration: adding, removing and reordering strategies, configuring fee receivers and rates, setting the vault implementation, and pointing the pool at its priority pool and rebase controller. Reward updates are gated on a designated `rewardsUpdater` and emergency pausing on a designated `emergencyPauser`, so the liveness of rebasing depends on those roles operating correctly.

The changes in this PR marginally increase the owner's influence over fee accounting rather than over principal. With the forced rebase removed, the timing of a fee change relative to the next rebase determines how much of the accrued reward period the new rate applies to, and that timing is at the owner's discretion. Total fees remain capped in code at 40% in `StakingPool` and 30% in each strategy, bounding the impact to a share of rewards. The practical recommendation is that fee changes be sequenced immediately after a rebase, and that the owner key be held by a multisig with timelocked upgrade authority.

6.2 Defensive Improvements

- Validate fee parameters at the point of configuration. `addFee` accepts a zero address receiver in `StakingPool`, `VaultControllerStrategy` and `PolygonStrategy`, and accepts a zero basis point amount in all four in-scope contracts, despite zero carrying a dedicated "remove this fee" meaning in `updateFee`. `EspressoStrategy` already validates the receiver and should be treated as the reference implementation.
- Guard the `totalStaked` update in `StakingPool::_updateStrategyRewards` against a negative result before the `uint256` cast. The current code is safe only by an invariant maintained independently by each strategy implementation, rather than by a check at the site that depends on it.
- Consider two-step ownership transfer and role separation between fee administration and strategy administration, given that a single owner key currently controls both.

6.3 Protocol Invariants

- The sum of all fee basis points must not exceed 4000 in `StakingPool` and 3000 in each strategy.
- `totalStaked` must always be greater than or equal to the sum of `totalDeposits` across all strategies, and must never be driven negative by an aggregate deposit change.

Summary

Project Name	PR 187
Repository	contracts
Commit	fd88dbcea0a2...
Fix Commit	bd1afd30dfa6...
Audit Timeline	Sep 21st - Sep 22nd, 2026
Methods	Manual Review

Issues Found

Severity	Found	Resolved	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	0	0	0
Informational	4	4	0
Gas Optimizations	0	0	0
Total	4	4	0

Summary of Findings

[I-1] Dead code in <code>StakingPool</code> <code>addFee</code> and <code>updateFee</code> after the removal of the strategy rewards update	Resolved
[I-2] Lack of <code>_feeBasisPoints</code> non zero validation in <code>addFee</code>	Resolved
[I-3] Unchecked <code>uint256</code> cast in <code>_updateStrategyRewards</code> silently corrupts <code>totalStaked</code> if negative rewards ever exceed it.	Resolved
[I-4] Lack of <code>_receiver</code> zero address validation in <code>addFee</code> and <code>updateFee</code> halts all reward updates until the owner corrects the entry	Resolved

7 Findings

7.1 Informational

7.1.1 Dead code in StakingPool addFee and updateFee after the removal of the strategy rewards update

Description: This PR removes the call to `_updateStrategyRewards` from the fee setters, so that changing a fee no longer triggers a rebase across all strategies.

In `StakingPool`, both `addFee` and `updateFee` built a `strategyIdxs` array containing the index of every strategy. That array existed for the sole purpose of being passed to `_updateStrategyRewards`. The call was removed, but the array construction that fed it was left behind.

contracts/core/StakingPool.sol#addFee

```
function addFee(address _receiver, uint256 _feeBasisPoints) external onlyOwner {
>>     uint256[] memory strategyIdxs = new uint256[](strategies.length);
>>     for (uint256 i = 0; i < strategyIdxs.length; ++i) {
>>         strategyIdxs[i] = i;
>>     }

    fees.push(Fee(_receiver, _feeBasisPoints));
    require(_totalFeesBasisPoints() <= 4000, "Total fees must be <= 40%");
}
```

contracts/core/StakingPool.sol#updateFee

```
function updateFee(
    uint256 _index,
    address _receiver,
    uint256 _feeBasisPoints
) external onlyOwner {
    require(_index < fees.length, "Fee does not exist");

>>     uint256[] memory strategyIdxs = new uint256[](strategies.length);
>>     for (uint256 i = 0; i < strategyIdxs.length; ++i) {
>>         strategyIdxs[i] = i;
>>     }

    if (_feeBasisPoints == 0) {
        fees[_index] = fees[fees.length - 1];
        fees.pop();
    } else {
        fees[_index].receiver = _receiver;
        fees[_index].basisPoints = _feeBasisPoints;
    }

    require(_totalFeesBasisPoints() <= 4000, "Total fees must be <= 40%");
}
```

`strategyIdxs` is written but never read in either function. The compiler allocates the memory and executes the loop regardless, since the array is populated through an index assignment that the optimizer does not eliminate.

Note that the equivalent cleanup was applied correctly elsewhere in this PR: the now-unused `_updateStrategyRewards` helper was deleted from both `EspressoStrategy` and `PolygonStrategy`. Only the `StakingPool` leftovers were missed.

However, the `_updateStrategyRewards` helper in `VaultControllerStrategy` is intentionally retained and is **not** dead code, because `OperatorVCS` still calls it in `removeVault` and `setOperatorRewardPercentage`.

Impact:

- Wasted gas on every fee change, scaling linearly with the number of strategies, for a loop whose result is discarded.

- Misleading code. A reader encountering the construction of a full strategy index list reasonably infers that the function still interacts with strategies, when it no longer does.

Recommended Mitigation: ove the strategyIdxs construction from both functions, matching the cleanup already performed in EspressoStrategy and PolygonStrategy.

```
function addFee(address _receiver, uint256 _feeBasisPoints) external onlyOwner {
-   uint256[] memory strategyIdxs = new uint256[](strategies.length);
-   for (uint256 i = 0; i < strategyIdxs.length; ++i) {
-       strategyIdxs[i] = i;
-   }
-
    fees.push(Fee(_receiver, _feeBasisPoints));
    require(_totalFeesBasisPoints() <= 4000, "Total fees must be <= 40%");
}
```

```
) external onlyOwner {
    require(_index < fees.length, "Fee does not exist");

-   uint256[] memory strategyIdxs = new uint256[](strategies.length);
-   for (uint256 i = 0; i < strategyIdxs.length; ++i) {
-       strategyIdxs[i] = i;
-   }
-
    if (_feeBasisPoints == 0) {
```

Stake.Link: Fixed in [29d1ba75](#).

Cyfrin: Verified.

7.1.2 Lack of _feeBasisPoints non zero validation in addFee

Description: Across the staking pool and every strategy, a fee is a pair of a receiver and a basis point value. The protocol treats a basis point value of zero as a sentinel meaning "remove this fee" rather than as a legitimate fee amount. This convention is established in updateFee, where passing zero deletes the entry instead of updating it.

contracts/core/StakingPool.sol#updateFee

```
function updateFee(
    uint256 _index,
    address _receiver,
    uint256 _feeBasisPoints
) external onlyOwner {
    require(_index < fees.length, "Fee does not exist");
    ...
>>   if (_feeBasisPoints == 0) {
>>       fees[_index] = fees[fees.length - 1];
>>       fees.pop();
    } else {
        fees[_index].receiver = _receiver;
        fees[_index].basisPoints = _feeBasisPoints;
    }

    require(_totalFeesBasisPoints() <= 4000, "Total fees must be <= 40%");
}
```

addFee does not enforce the same convention. It pushes whatever value it is given, including zero, with no validation.

contracts/core/StakingPool.sol#addFee

```
function addFee(address _receiver, uint256 _feeBasisPoints) external onlyOwner {
    ...
```

```
>>     fees.push(Fee(_receiver, _feeBasisPoints));
        require(_totalFeesBasisPoints() <= 4000, "Total fees must be <= 40%");
    }
```

The same issue is present in all three strategies, each of which applies the identical zero as removal convention in its own `updateFee`:

contracts/espressoStaking/EsspressoStrategy.sol#addFee

```
function addFee(address _receiver, uint256 _feeBasisPoints) external onlyOwner {
    if (_receiver == address(0)) revert InvalidAddress();
>>     fees.push(Fee(_receiver, _feeBasisPoints));
        if (_totalFeesBasisPoints() > 3000) revert FeesTooLarge();
        emit AddFee(_receiver, _feeBasisPoints);
}
```

contracts/polygonStaking/PolygonStrategy.sol#addFee

```
function addFee(address _receiver, uint256 _feeBasisPoints) external onlyOwner {
>>     fees.push(Fee(_receiver, _feeBasisPoints));
        if (_totalFeesBasisPoints() > 3000) revert FeesTooLarge();
        emit AddFee(_receiver, _feeBasisPoints);
}
```

contracts/linkStaking/base/VaultControllerStrategy.sol#addFee

```
function addFee(address _receiver, uint256 _feeBasisPoints) external onlyOwner {
>>     fees.push(Fee(_receiver, _feeBasisPoints));
        if (_totalFeesBasisPoints() > 3000) revert FeesTooLarge();
}
```

Adding a fee of zero basis points creates an entry that pays nothing but still occupies a slot in the `fees` array. Because the array is compacted with a swap and pop in `updateFee`, the index of an existing entry is not stable, which makes a stale zero entry awkward to reason about and easy to mistarget when the owner later attempts to clean it up.

Impact:

- The two functions `add` and `update fee` disagree about what a zero basis point value means. `addFee` accepts it as a fee, `updateFee` interprets it as a removal instruction.

Recommended Mitigation: Reject a zero basis point value in `addFee` so that the meaning of zero stays confined to the removal path in `updateFee`.

Stake.Link: Fixed in [f2d3f2e6](#).

Cyfrin: Verified.

7.1.3 Unchecked uint256 cast in `_updateStrategyRewards` silently corrupts `totalStaked` if negative rewards ever exceed it.

Description: `StakingPool` tracks the total amount of staked asset tokens in `totalStaked`. When rewards are updated, the net deposit change reported by the selected strategies is applied to it directly.

contracts/core/StakingPool.sol#_updateStrategyRewards

```
// update totalStaked if there was a net change in deposits
if (totalRewards != 0) {
>>     totalStaked = uint256(int256(totalStaked) + totalRewards);
}
```

`totalRewards` is a signed value and is expected to be negative whenever a strategy reports a loss, such as a slashing event. The expression is therefore performing signed arithmetic and then casting the result back to an unsigned type.

Under Solidity 0.8.x, the addition `int256(totalStaked) + totalRewards` is checked arithmetic, but a negative sum is a perfectly valid `int256` and does not overflow. The subsequent `uint256(...)` conversion is a bit reinterpretation, not a range checked conversion, so a negative result does not revert. It wraps to an enormous positive number.

If `totalStaked` were 1 and `totalRewards` were -15, the result would be:

```
uint256(int256(1) + (-15)) == uint256(int256(-14))
                          == 0xffff...ff2
                          == 1157920892373161954235709850086879078532699846656405640394575840079131296
                          ↪ 39922
```

`totalStaked` is the backing value for the entire liquid staking token. It is returned directly by `totalSupply`, and it is the numerator used to convert shares into balances.

contracts/core/base/StakingRewardsPool.sol

```
>> function totalSupply() public view override returns (uint256) {
    return _totalStaked();
}

function getStakeByShares(uint256 _amount) public view returns (uint256) {
    if (totalShares == 0) {
        return _amount;
    } else {
>>     return (_amount * _totalStaked()) / totalShares;
    }
}
```

A corrupted `totalStaked` would therefore make every holder balance read as an astronomically large number, break `getSharesByStake` in the opposite direction, and propagate into the fee share calculation in the same function, which divides by `totalStaked - totalFeeAmounts`. There is no administrative setter for `totalStaked`, so recovery would require a contract upgrade.

There is no clear path that reaches this state in the current codebase. When all strategies handle deposits and the maximum slashing is 100% (total loss), no strategy can produce a negative `depositChange` that exceeds `totalStaked` for that strategy unless an attack occurred within the strategy itself.

The issue can occur if one of the strategies gets attacked, or new strategies are added that can result in loss exceeds deposit (like a debt mechanism)

Impact:

- Breaking the staking system and share value calculations for the protocol

Recommended Mitigation: Validate the result before casting, where if `totalRewards < 0` we make sure that the `Math.ads(totalRewards)` is smaller than `totalStaked`

Stake.Link: Fixed in [964190f4](#).

Cyfrin: Verified.

7.1.4 Lack of `_receiver` zero address validation in `addFee` and `updateFee` halts all reward updates until the owner corrects the entry

Description: Fee receivers are paid by minting liquid staking tokens to the pool and then transferring them out to each receiver in turn. The transfer is performed with `transferAndCallFrom` inside the fee distribution loop of `StakingPool::_updateStrategyRewards`.

contracts/core/StakingPool.sol#_updateStrategyRewards

```

// distribute fees to receivers if there are any
if (totalFeeAmounts > 0) {
    uint256 sharesToMint = (totalFeeAmounts * totalShares) /
        (totalStaked - totalFeeAmounts);
    _mintShares(address(this), sharesToMint);

    uint256 feesPaidCount;
    for (uint256 i = 0; i < receivers.length; i++) {
        for (uint256 j = 0; j < receivers[i].length; j++) {
            if (feesPaidCount == totalFeeCount - 1) {
                transferAndCallFrom(
                    address(this),
                    receivers[i][j],
                    balanceOf(address(this)),
                    "0x"
                );
            } else {
                transferAndCallFrom(address(this), receivers[i][j], feeAmounts[i][j], "0x");
                feesPaidCount++;
            }
        }
    }
}

```

transferAndCallFrom delegates to _transfer, which rejects the zero address unconditionally.

contracts/core/base/StakingRewardsPool.sol#_transfer

```

function _transfer(address _sender, address _recipient, uint256 _amount) internal override {
    uint256 sharesToTransfer = getSharesByStake(_amount);

    require(_sender != address(0), "Transfer from the zero address");
    require(_recipient != address(0), "Transfer to the zero address");
    require(sharesToTransfer != 0, "Transfer amount too small");
    require(shares[_sender] >= sharesToTransfer, "Transfer amount exceeds balance");
}

```

A fee receiver set to the zero address therefore causes an unconditional revert inside the distribution loop. The loop has no error handling, so the revert propagates and the whole reward update transaction fails.

EspressoStrategy guards against this on both entry points.

contracts/espressoStaking/EspressoStrategy.sol#addFee

```

function addFee(address _receiver, uint256 _feeBasisPoints) external onlyOwner {
    if (_receiver == address(0)) revert InvalidAddress();
    fees.push(Fee(_receiver, _feeBasisPoints));
    if (_totalFeesBasisPoints() > 3000) revert FeesTooLarge();
    emit AddFee(_receiver, _feeBasisPoints);
}

```

The remaining three implementations do not. StakingPool, which is the contract that actually performs the transfer, accepts any receiver:

contracts/core/StakingPool.sol#addFee

```

function addFee(address _receiver, uint256 _feeBasisPoints) external onlyOwner {
    ...
    fees.push(Fee(_receiver, _feeBasisPoints));
    require(_totalFeesBasisPoints() <= 4000, "Total fees must be <= 40%");
}

```

contracts/linkStaking/base/VaultControllerStrategy.sol#addFee

```
function addFee(address _receiver, uint256 _feeBasisPoints) external onlyOwner {
>>     fees.push(Fee(_receiver, _feeBasisPoints));
        if (_totalFeesBasisPoints() > 3000) revert FeesTooLarge();
    }
}
```

contracts/polygonStaking/PolygonStrategy.sol#addFee

```
function addFee(address _receiver, uint256 _feeBasisPoints) external onlyOwner {
>>     fees.push(Fee(_receiver, _feeBasisPoints));
        if (_totalFeesBasisPoints() > 3000) revert FeesTooLarge();
        emit AddFee(_receiver, _feeBasisPoints);
    }
}
```

The same omission applies to the update branch of updateFee in all three, which assigns fees[_index].receiver = _receiver without validation.

Impact:

- Failed to distribute fees because of incorrect configuration.

Recommended Mitigation: Validate _receiver in addFee and in the update branch of updateFee, matching the check already present in EspressoStrategy.

Stake.Link: Fixed in [6fd0535d](#).

Cyfrin: Verified.